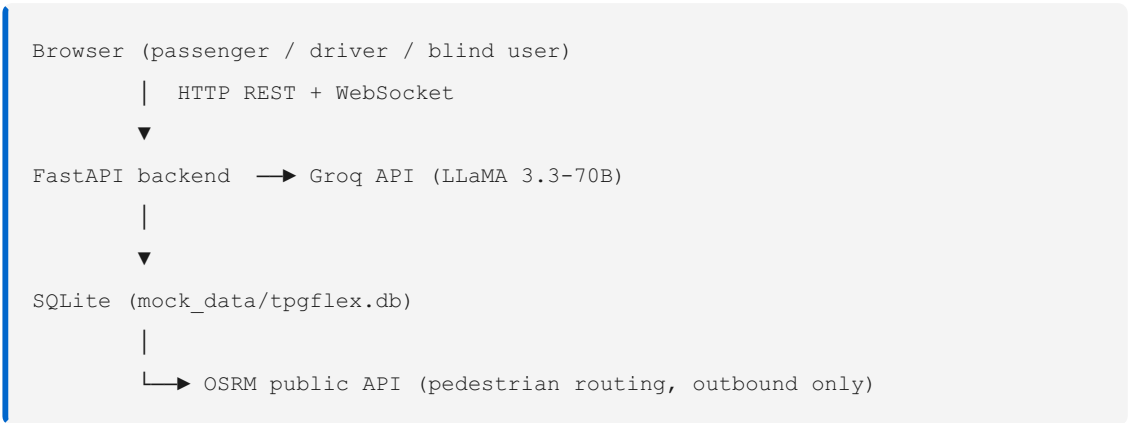


tpgFlex Voice Booking — Architecture

Overview

tpgFlex Voice Booking is an AI-powered ride-booking assistant for tpgFlex, Geneva's on-demand shared minibus service. Users speak or type a destination; the system extracts intent, confirms details, and creates a booking. A separate driver panel handles real-time accessibility alerts. A blind navigation mode provides audio-guided pedestrian routing.



Frontend — 5 Pages

All pages are static HTML/CSS/JS served by the backend at `/static/<page>.html` . They must be loaded through `http://localhost:8000/` (not as `file://` URLs) because navigation links use absolute paths like `/static/navigate_user.html` .

File	Purpose	Who uses it
<code>index.html</code>	Main passenger UI: voice/text input, chat, map, booking flow	Passenger
<code>driver_panel.html</code>	Real-time accessibility alerts, ramp request queue	Driver
<code>navigate_user.html</code>	Audio-guided step-by-step navigation (blind mode, user side)	Blind passenger
<code>navigate_display.html</code>	Mirrored map display for a companion or mounted screen	Companion/ display
<code>navigate_map.html</code>	Map-only component embedded by navigate pages	Internal

The main page (`index.html`) talks to the backend over both REST (booking) and WebSocket (navigation relay). The driver panel connects to `/ws/driver` and listens for push messages. The navigate pages connect to `/ws/navigate/{role}` where role is `user` or `display`.

Backend — FastAPI (`backend/main.py`)

Single-process FastAPI app running under uvicorn on port 8000.

REST Endpoints

Voice & booking

Method	Path	What it does
POST	<code>/voice/process</code>	Core endpoint. Takes user text + history, calls AI, runs booking logic
POST	<code>/quick-book</code>	Tap-to-book shortcut — no AI, no confirmation loop

Data queries

Method	Path	What it does
GET	<code>/stops</code>	List all active stops
GET	<code>/stops/nearest?lat&lon</code>	Nearest stop to a GPS coordinate
GET	<code>/api/stops/nearby?lat&lng</code>	Top 5 stops sorted by distance (for navigation)
GET	<code>/bookings/{user_id}</code>	Last 10 bookings for a user
GET	<code>/vehicles</code>	List all vehicles
GET	<code>/health</code>	Health check

Ramp / accessibility

Method	Path	What it does
POST	<code>/ramp-request</code>	Create a wheelchair ramp request; pushes to all driver panels
GET	<code>/ramp-request/{req_id}</code>	Get status of a specific ramp request
POST	<code>/ramp-request/{req_id}/acknowledge</code>	Driver confirms ramp is deployed
GET	<code>/ramp-requests/pending</code>	All unacknowledged ramp requests

Routing & static

Method	Path	What it does
GET	/api/route	Proxy to OSRM; enriches steps with blind-navigation hints
GET	/	Serves frontend/index.html
GET	/static/*	Serves all frontend files

WebSocket Endpoints

Path	Purpose
/ws/driver	Persistent connection for each driver panel. Backend pushes ramp_request and ramp_acknowledged events in real time
/ws/navigate/{role}	Relay bridge between user and display sides of blind navigation. Any message received from one role is forwarded verbatim to the other

In-Memory State

Variable	Type	Contents
_booking_context	dict[user_id → dict]	Per-user conversation context: cached origin, destination, time, passengers across multi-turn dialogue
_ramp_requests	dict[req_id → dict]	All ramp requests for the session (not persisted to DB)
_driver_clients	list[WebSocket]	All currently connected driver panel sockets
nav_clients	dict	{"user": ws, "display": ws} — the active navigation pair

AI Agent Layer (backend/agent/)

Intent Detection (intent.py)

1. Builds a message list: system prompt + conversation history + current user message.
2. Sends to Groq API (llama-3.3-70b-versatile , temperature 0.1, max 400 tokens).
3. Parses the JSON response. Strips markdown fences if the model wraps them.
4. On any failure (timeout, bad JSON, empty response) falls back to a local keyword parser.

Groq response schema:

```
{
  "intent": "book_ride | awaiting_confirmation | confirmed | find_stop | check_status | greeting | re
```

```

"origin": "<stop name or null>",
"destination": "<stop name or null>",
"time": "<HH:MM or now or null>",
"passengers": 1,
"reply": "<short spoken reply>",
"ready_to_book": false
}

```

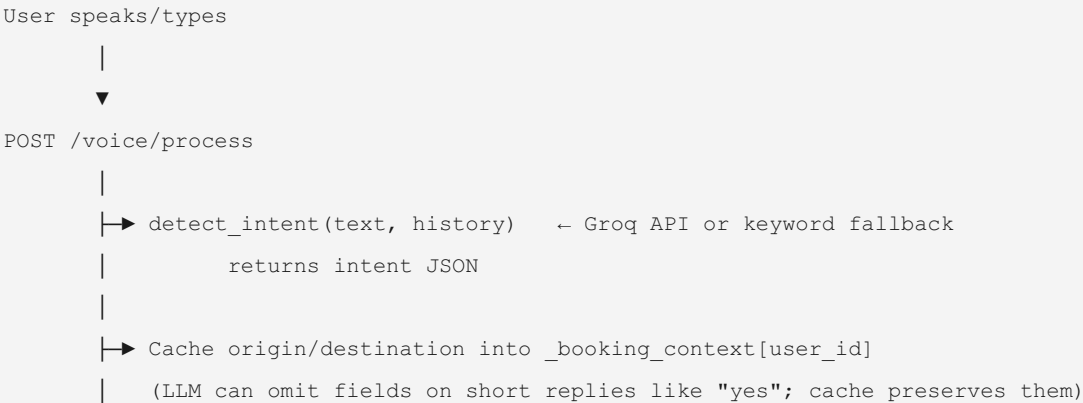
Keyword fallback (`_keyword_intent`): scans the user message and history for known place keywords (e.g. "airport" → Palexpo, "hospital" → Hôpital Cantonal) and constructs a minimal intent dict. Used silently when Groq is unavailable.

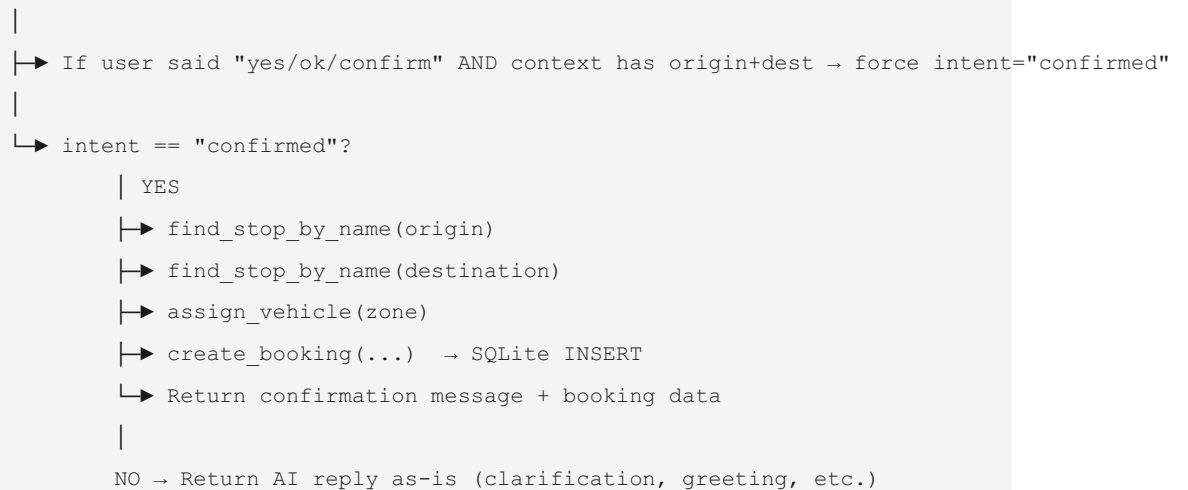
Booking Engine (`booking_engine.py`)

Called by `main.py` after the AI returns `confirmed` intent.

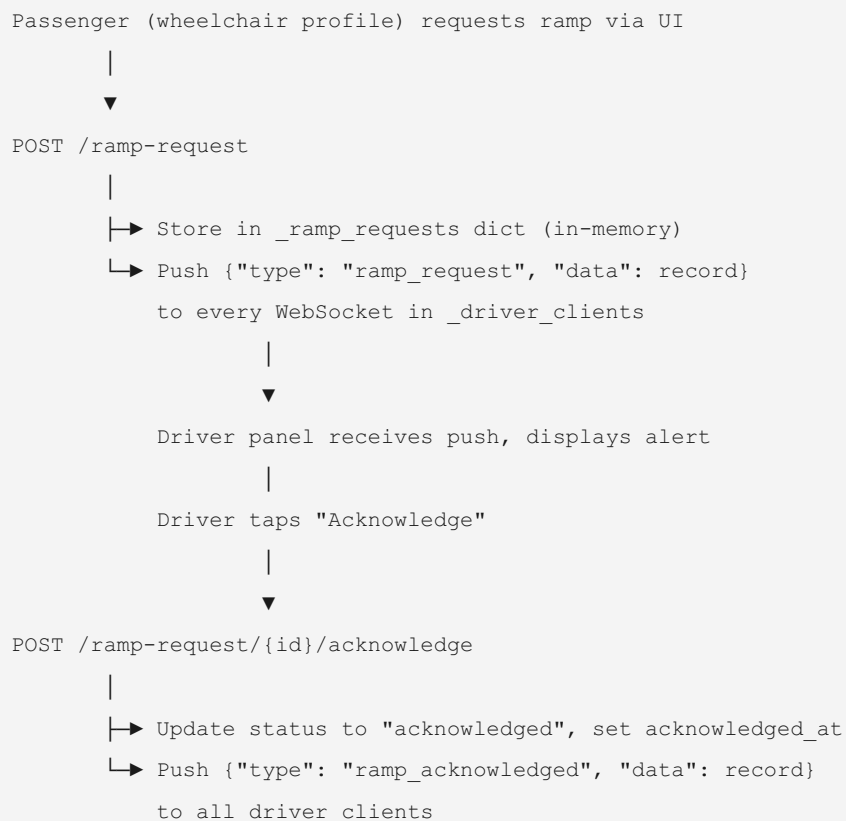
Function	What it does
<code>find_stop_by_name(query)</code>	Fuzzy-matches a free-text name to a stop using <code>difflib.get_close_matches</code> (cutoff 0.3), with a substring fallback
<code>nearest_stop_to_coords(lat, lon)</code>	Returns the stop with the smallest haversine distance
<code>assign_vehicle(zone)</code>	Finds an <code>available</code> vehicle, optionally filtered by zone. If none found (all <code>in_service</code>), recycles them back to <code>available</code>
<code>create_booking(...)</code>	Inserts a booking row, marks the vehicle <code>in_service</code> , returns a summary dict
<code>get_user_history(user_id)</code>	Returns last N bookings for a user

Conversation Flow (Voice Booking)





Ramp Request Flow (Accessibility)



Blind Navigation Flow



```
|
backend relay (nav_clients dict)
any message from "user" → forwarded to "display"
any message from "display" → forwarded to "user"
```

Step-by-step routing:

```
navigate_user.html
|
└─ GET /api/route?from_lat&from_lng&to_lat&to_lng
    |
    └─ OSRM public API (foot profile, steps+geometry)
        |
        _enrich_pedestrian_steps()
        adds per-step fields:
        - is_crossing (bool)
        - crossing_hint (spoken warning at junctions)
        - side_streets (count of intersections mid-segment)
        - path_hint (reassurance for long straight segments)
        |
        Returns enriched JSON to frontend
        Frontend reads steps aloud via Web Speech API (TTS)
```

Database (`backend/mock_data/tpgflex.db`)

SQLite. Seeded once by `backend/mock_data/seed.py` .

Table	Key columns	Records
<code>stops</code>	<code>stop_id</code> , <code>name</code> , <code>latitude</code> , <code>longitude</code> , <code>zone</code> , <code>is_active</code>	15
<code>vehicles</code>	<code>vehicle_id</code> , <code>status</code> , <code>capacity</code> , <code>zone</code> , <code>current_lat/lng</code>	8
<code>users</code>	<code>user_id</code> , <code>name</code> , <code>email</code> , <code>phone</code> , <code>preferred_language</code>	5
<code>bookings</code>	<code>booking_id</code> , <code>status</code> , <code>passenger_status</code> , <code>user_id</code> , <code>pickup_id</code> , <code>dropoff_id</code> , <code>selected_pickup_time</code> , <code>vehicle_id</code> , <code>passengers</code>	118 (seeded)
<code>locations</code>	<code>date</code> , <code>driver</code> , <code>latitude</code> , <code>longitude</code> , <code>bearing</code> , <code>accuracy</code>	80 (vehicle telemetry)

Vehicle status lifecycle: `available` → `in_service` (on booking) → recycled back to `available` when the pool is exhausted.

Sample Data

stops

stop_id	name	latitude	longitude	zone	is_active
STOP_001	Cornavin	46.2100	6.1422	A	1
STOP_002	Carouge-Marché	46.1833	6.1400	B	1
STOP_003	Palexpo	46.2333	6.1167	C	1
STOP_004	Hôpital Cantonal	46.1938	6.1477	A	1
STOP_005	CERN	46.2330	6.0550	C	1

vehicles

vehicle_id	status	capacity	zone	current_lat	current_lng
VEH_001	in_service	8	A	46.2100	6.1422
VEH_002	available	6	B	46.1833	6.1400
VEH_003	available	8	C	46.2333	6.1167
VEH_004	in_service	8	A	46.1938	6.1477

users

user_id	name	email	phone	preferred_language
USR_001	Marie Dubois	marie.dubois@email.ch	+41791234567	fr
USR_002	John Smith	john.smith@email.com	+41791234568	en
USR_003	Ahmed Hassan	ahmed.h@email.com	+41791234569	fr

bookings

booking_id	status	passenger_status	user_id	pickup	dropoff	pickup_time
33c59378	Validated	Trip completed	USR_002	Palexpo	Cornavin	2026-04-06T08:30:00
5d192941	Validated	Trip completed	USR_004	Bachet-de-Pesay	Vernier-Village	2026-03-30T14:15:00
0abd7a6a	Cancelled	Cancelled	USR_001	Onex-Cité	Hôpital Cantonal	2026-03-28T05:00:00
8a03e037	Cancelled	Cancelled	USR_002	Carouge-Marché	Aire	2026-03-21T06:00:00

locations (live vehicle telemetry)

location_id	datetime	driver	latitude	longitude	bearing	accuracy
6e7d3c8c	2026-04-18T20:38:38	Driver_VEH_001	46.20931	6.14494	8.55	86.06
3c6de40f	2026-04-18T21:02:38	Driver_VEH_001	46.20823	6.14648	8.60	259.68
e5e6dc82	2026-04-18T20:05:38	Driver_VEH_001	46.21329	6.14157	3.42	127.89

External Dependencies

Service	Used for	Failure mode
Groq API (<code>api.groq.com</code>)	LLM intent extraction	Silent fallback to keyword parser
OSRM (<code>router.project-osrm.org</code>)	Pedestrian turn-by-turn routing	Returns <code>{"error": "routing failed"}</code>

Running the Project

```
# Install dependencies (one time)
pip install -r requirements.txt

# Start backend (keep running)
uvicorn backend.main:app --reload --port 8000

# Open app - must use the backend URL, not a file:// path
open http://localhost:8000/
```

API explorer: `http://localhost:8000/docs`